Faculty Teaching & Office-Hours Scheduler

Project Report

COSC 369 – Software Engineering I, Project 1

Implementation: C (static library + executable), Python, SWI-Prolog via PySwip

October 12, 2026

Contents

1. Executive Summary.....
2. Project Objectives and Requirements.....
3. System Architecture.....
3.1 Directory Layout.....
4. C Module Design.....
4.1 Data Representation.....
4.2 Input Parsing and Validation.....
5. Office-Hours Scheduling Algorithm.....
5.1 Pass 1 – Around Teaching Days.....
5.2 Pass 2 – Preferred-Day Fallback.....
5.3 Correctness Properties.....
6. Build System and Static Library.....
7. Input and Output Files.....
7.1 Inputs.....
7.2 Outputs.....
8. Results.....
8.1 Summary Statistics.....
9. Prolog Knowledge and Question System.....
10. Continuous Integration.....
11. Observations and Limitations.....
12. Recommendations for Future Work.....
13. Conclusion.....

1. Executive Summary

This project automates the creation of faculty teaching schedules and conflict-free weekly office-hour schedules. A C program reads two pipe-delimited input files – a class schedule and a faculty information roster – and produces, for every instructor, a teaching table and a set of office-hour blocks that satisfy that instructor's required weekly office hours without ever overlapping a class. Python helpers turn the program's CSV output into a formatted Excel workbook and into a Prolog knowledge base, which a natural-language question interface queries through PySwip. A GitLab CI/CD pipeline rebuilds and verifies the entire chain on every push.

Key results from the current run on the bundled CTEC Fall 2026 data:

- **54 course sections** (15 online) taught by **27 faculty entries** were loaded with no parse errors.
- **67 office-hour blocks** were generated, totalling **118 hours** against a combined requirement of 118 hours.
- Every faculty member's weekly office-hour requirement was met in full; no office-hour block overlaps that instructor's class times.
- 4 instructors needed the preferred-day fallback because their teaching-day gaps could not absorb all required hours.
- The generated Prolog knowledge base contains 656 unique facts, combining schedule data with the COSC 369 syllabus.

2. Project Objectives and Requirements

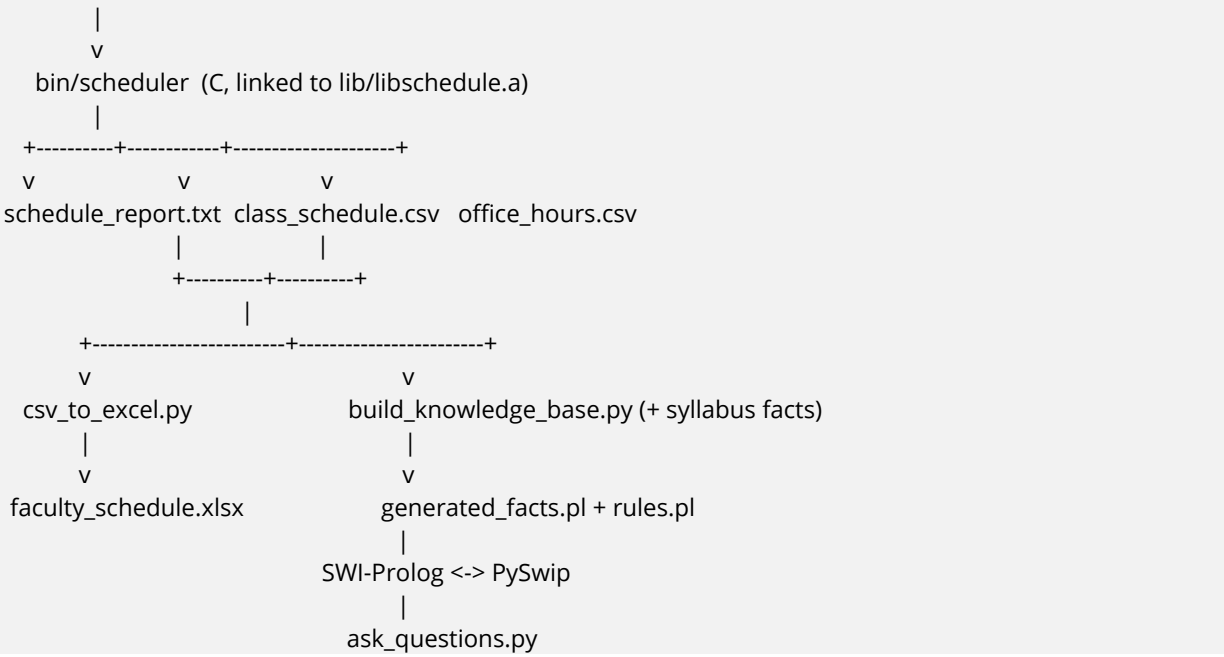
The project handout asks for a program that, given a class schedule and faculty information, generates each instructor's teaching schedule and office-hour schedule in the style of the sample office-hours table. The implementation addresses the following requirements:

1. Read the class schedule (syllabus data) and faculty information (office-hour policy data) from input files.
2. Generate each instructor's teaching schedule.
3. Generate an office-hour schedule that meets each instructor's required weekly hours and never conflicts with class times.
4. Package the reusable logic as a static library and build it with a Makefile.
5. Produce human-readable output (text report and Excel workbook).
6. Extension: expose the schedule and syllabus data through a Prolog knowledge base and a question-answering interface.
7. Extension: verify the build and outputs automatically with GitLab CI/CD.

3. System Architecture

The system is a pipeline of three layers. The C layer owns all schedule computation; Python is purely a presentation and conversion layer; Prolog provides reasoning and querying over the results.

```
class_schedule.txt + faculty_info.txt
```



3.1 Directory Layout

Path	Purpose
include/	Public headers: course.h, faculty.h, parser.h, scheduler.h
src/	Module implementations; main.c is the entry point, the rest form the library
data/	Input files: class_schedule.txt, faculty_info.txt, syllabi/
scripts/	Python helpers: csv_to_excel.py, build_knowledge_base.py, ask_questions.py
prolog/	rules.pl (hand-written) and generated_facts.pl (generated)
build/, lib/, bin/	Object files, libschedule.a and the scheduler executable (generated)
output/	Text report, two CSVs and the Excel workbook (generated)
Makefile, .gitlab-ci.yml	Build automation and CI/CD pipeline

4. C Module Design

The C code is split into four library modules plus a driver. Each module has a single responsibility and a matching header in include/.

Module	Responsibility	Key functions / types
course.c / course.h	Course data type; day-bitmask and time conversion helpers	Course, CourseList, parse_days, time_to_minutes, minutes_to_time_str, course_meets_on
faculty.c / faculty.h	Faculty data type and lookup	Faculty, FacultyList, find_faculty_by_name
parser.c / parser.h	Reads and validates the two pipe-delimited	load_course_list, load_faculty_list

Module	Responsibility	Key functions / types
	input files	
scheduler.c / scheduler.h	Office-hour generation and all output writers	generate_office_schedules, write_report, write_csv_summary, write_csv_courses
main.c	Argument handling, validation warnings, output directory creation, orchestration	main, ensure_dir

4.1 Data Representation

- **Days** are stored as a 5-bit mask (DAY_MON = 0x01 … DAY_FRI = 0x10), parsed from letter codes M, T, W, R, F. This makes "does this course meet on day d" a single bitwise AND.
- **Times** are stored as integer minutes since midnight, so gap and overlap arithmetic is plain integer subtraction. They are formatted as 12-hour "h:MM AM/PM" only on output.
- **Collections** are fixed-size arrays (MAX_COURSES = 200, MAX_FACULTY = 100, MAX_OFFICE_BLOCKS = 20 per instructor), declared static in main to avoid large stack frames and any dynamic allocation.

4.2 Input Parsing and Validation

Both loaders skip blank lines and # comments, split each line on "|", and trim whitespace. Lines with the wrong field count, invalid days or times, an end time not after the start time, or non-positive weekly office hours are reported to stderr with the file name and line number and are skipped rather than aborting the run. After loading, main warns about any instructor in the class schedule who has no faculty entry, and about any faculty member whose requirement could not be fully scheduled.

5. Office-Hours Scheduling Algorithm

Office hours are placed inside an 8:00 AM – 5:00 PM working window, in blocks of up to 2 hours (DEFAULT_BLOCK = 120 minutes). Gaps shorter than 30 minutes (MIN_BLOCK) are ignored. For each faculty member the scheduler works in two passes:

5.1 Pass 1 – Around Teaching Days

For each weekday the instructor teaches, their classes for that day are collected and sorted by start time. Then:

- **Before the first class:** if there is at least 30 minutes between 8:00 AM and the first class, a block ending exactly when that class begins is added. This is what gives evening-only instructors morning or afternoon availability.
- **Between classes:** if a gap is at least 4 hours, one block is placed immediately after the earlier class and, if hours remain, a second block immediately before the next class. Smaller gaps get a single block right after the earlier class.
- **After the last class:** the gap up to 5:00 PM is used the same way.

Every block records the course it is adjacent to, so students can see which section an office hour is meant to support. Placement stops as soon as the weekly requirement is met.

5.2 Pass 2 – Preferred-Day Fallback

If hours remain after Pass 1, the scheduler walks the instructor's PreferredDays, skipping days they already teach on, and adds a general block starting at 8:00 AM on each until the requirement is met. These blocks have no related course and appear as "General" in the workbook.

5.3 Correctness Properties

- No overlap with classes: every block lies strictly inside a gap computed from that instructor's own sorted class times.
- Requirement bound: each block is capped at the remaining minutes, so an instructor is never scheduled more hours than required.
- Determinism: identical inputs always produce identical outputs, which is what lets CI compare runs.

6. Build System and Static Library

The Makefile compiles course.c, faculty.c, parser.c and scheduler.c into build/*.o, archives them with ar rcs into lib/libschedule.a, then compiles main.c and links it against the archive (-Llib -lschedule) to produce bin/scheduler. Compiler flags are -Wall -Wextra -std=c11 -Iinclude -g. Pattern rules and order-only prerequisites mean directories are created on demand and only changed sources are rebuilt.

Target	Action
make / make all	Build lib/libschedule.a and bin/scheduler
make run	Run the scheduler on the sample data, then build the Excel workbook
make knowledge	make run, then generate prolog/generated_facts.pl
make demo	make knowledge, then ask a fixed set of demo questions (non-interactive)
make ask	make knowledge, then start the interactive question system
make docs	Alias for make run
make clean	Remove build/, lib/, bin/, output/ and the generated facts file

The Python interpreter is overridable (PYTHON ?= python), e.g. `make run PYTHON=python3` on macOS/Linux or `make run PYTHON=py` on Windows.

7. Input and Output Files

7.1 Inputs

File	Format
data/class_schedule.txt	CourseCode|CourseTitle|Instructor|Days|StartTime|EndTime|Room (24-hour times; R = Thursday)
data/faculty_info.txt	Instructor|Department|WeeklyOfficeHours|OfficeRoom|PreferredDays
data/syllabi/appolo_tankeh_syllabus_facts.txt	Typed records (COURSE, INSTRUCTOR, EMAIL, OFFICE, OFFICE_HOURS, PREREQUISITE, DESCRIPTION, POLICY, EVENT) – used only by the knowledge system

The class schedule is adapted from the CTEC Fall 2026 schedule and lists every section for every instructor. Two purely asynchronous online sections with no meeting time are intentionally omitted, since there is no class time to schedule around. Weekly office hours are set at roughly two hours per section taught, with a minimum of three.

7.2 Outputs

File	Producer	Contents
output/schedule_report.txt	C	Per-instructor class table and Monday–Friday office-hour grid
output/class_schedule.csv	C	One row per course section
output/office_hours.csv	C	One row per office-hour block with its related course
output/faculty_schedule.xlsx	csv_to_excel.py	One formatted sheet per instructor: CLASS SCHEDULE and OFFICE HOURS tables
prolog/generated_facts.pl	build_knowledge_base.py	656 unique Prolog facts

8. Results

The table below summarizes the current output for all 27 faculty entries. "Scheduled" is computed from output/office_hours.csv. Blocks marked "General" come from the preferred-day fallback.

Instructor	Office	Sections	Req. hrs	Sched. hrs	Office-hour blocks
Adedoyin, Anthony	CS201	3	6	6	Mon 2:00 PM–4:00 PM (CTEC350.170) Tue 8:00 AM–10:00 AM (General) Wed 2:00 PM–4:00 PM (CTEC350.170)
Agboli, P.	CS202	2	4	4	Mon 3:00 PM–5:00 PM (CTEC120.171) Wed 3:00 PM–5:00 PM (CTEC120.171)
Baptiste, R.	CS203	3	6	6	Mon 8:00 AM–10:00 AM (General) Tue 10:00 AM–12:00 PM (CTEC214.180)

Instructor	Office	Sections	Req. hrs	Sched. hrs	Office-hour blocks
					Thu 10:00 AM–12:00 PM (CTEC214.180)
Bemley	CS204	1	3	3	Tue 10:00 AM–12:00 PM (CTEC128.170) Tue 2:50 PM–3:50 PM (CTEC128.170)
Bemley, J	CS205	1	3	3	Tue 2:00 PM–4:00 PM (CTEC298.180) Thu 3:00 PM–4:00 PM (CTEC298.180)
Bemley, Jesse	CS206	1	3	3	Tue 2:00 PM–4:00 PM (CTEC671.555) Thu 3:00 PM–4:00 PM (CTEC671.555)
Agada, Ruth	CS207	3	6	6	Mon 3:00 PM–5:00 PM (CTEC711.555) Mon 5:50 PM–7:00 PM (CTEC711.555) Wed 3:00 PM–5:00 PM (CTEC711.555) Wed 5:50 PM–6:40 PM (CTEC711.555)
Bright, Marvin	CS208	2	4	4	Mon 5:00 PM–7:00 PM (CTEC720.555) Tue 5:00 PM–7:00 PM (CTEC721.555)
Carter, Adrian D.	CS209	3	6	6	Mon 2:00 PM–4:00 PM (CTEC445.170) Tue 5:00 PM–7:00 PM (CTEC424.555) Wed 2:00 PM–4:00 PM (CTEC445.170)
Chery, Pierrot	CS210	3	6	6	Mon 8:00 AM–10:00 AM (General) Tue 2:00 PM–4:00 PM (CTEC305.170) Thu 2:00 PM–4:00 PM (CTEC305.170)
Deal, Shawn	CS211	1	3	3	Tue 5:00 PM–7:00 PM (CTEC226.180) Thu 6:00 PM–7:00 PM (CTEC226.180)
Dixon, Carl	CS212	2	4	4	Mon 2:00 PM–4:00 PM (CTEC294.170) Wed 2:00 PM–4:00 PM (CTEC294.170)
Dr. Stone, D. B.	CS213	2	4	4	Mon 10:00 AM–12:00 PM (CTEC120.170) Mon 2:00 PM–4:00 PM (CTEC120.170)
Harris, Eugene T.	CS214	2	4	4	Tue 10:50 AM–12:50 PM (CTEC111.172) Tue 2:00 PM–4:00 PM (CTEC114.172)
Francis, Wilbert R.	CS215	1	3	3	Mon 5:00 PM–7:00 PM (CTEC475.180) Wed 6:00 PM–7:00 PM (CTEC475.180)
Cooper, Henri E.	CS216	1	3	3	Mon 2:00 PM–4:00 PM (CTEC111.173) Wed 3:00 PM–4:00 PM (CTEC111.173)
Bengesi, Staphord	CS217	3	6	6	Mon 10:50 AM–12:50 PM (CTEC111.171) Mon 2:00 PM–4:00 PM (CTEC445.180) Wed 10:50 AM–12:50 PM (CTEC111.171)
Lamar, Courtney	CS218	1	3	3	Mon 2:00 PM–4:00 PM (CTEC393.555) Wed 3:00 PM–4:00 PM (CTEC393.555)
Miller, Stephanie R.	CS219	1	3	3	Mon 5:00 PM–7:00 PM (CTEC426.170) Wed 6:00 PM–7:00 PM (CTEC426.170)
Njeh, Felix	CS220	1	3	3	Mon 2:00 PM–4:00 PM (CTEC651.555) Wed 3:00 PM–4:00 PM (CTEC651.555)
Okonofua, Henry	CS221	2	4	4	Tue 2:00 PM–4:00 PM (CTEC410.170) Thu 2:00 PM–4:00 PM (CTEC410.170)

Instructor	Office	Sections	Req. hrs	Sched. hrs	Office-hour blocks
Roberts, Curtis	CS222	3	6	6	Mon 8:00 AM–10:00 AM (General) Tue 2:00 PM–4:00 PM (CTEC335.170) Thu 2:00 PM–4:00 PM (CTEC335.170)
Tolbert, Brett	CS223	2	4	4	Mon 3:00 PM–5:00 PM (CTEC345.170) Wed 3:00 PM–5:00 PM (CTEC345.170)
Dali, Loubna	CS224	2	4	4	Mon 5:00 PM–7:00 PM (CTEC450.180) Tue 8:00 AM–9:00 AM (CTEC111.170) Tue 11:50 AM–12:50 PM (CTEC111.170)
Vega, J.	CS225	3	6	6	Mon 9:00 AM–11:00 AM (CTEC220.170) Mon 1:50 PM–3:50 PM (CTEC220.180) Wed 9:00 AM–11:00 AM (CTEC220.170)
Lee, Victoria	CS226	1	3	3	Mon 2:00 PM–4:00 PM (CTEC114.170) Wed 3:00 PM–4:00 PM (CTEC114.170)
Wilson, Marcella J.	CS227	4	8	8	Mon 2:00 PM–4:00 PM (CTEC293.170) Tue 8:00 AM–10:00 AM (CTEC415.180) Tue 2:40 PM–4:00 PM (CTEC230.170) Wed 2:00 PM–4:00 PM (CTEC293.170) Thu 9:20 AM–10:00 AM (CTEC415.180)

8.1 Summary Statistics

Measure	Value
Course sections loaded	54
Online sections	15
Faculty entries loaded	27
Office-hour blocks generated	67
Total required office hours / week	118
Total scheduled office hours / week	118
Instructors with requirement fully met	27 of 27
Instructors using preferred-day fallback	4 (Adedoyin, Anthony; Baptiste, R.; Chery, Pierrot; Roberts, Curtis)

9. Prolog Knowledge and Question System

build_knowledge_base.py converts the two CSVs, the faculty file and the syllabus facts file into Prolog facts such as teaches/2, class_meets/4, office_hour/5, department/2, office_room/2, required_office_hours/2, preferred_day/2, prerequisite/2, syllabus_policy/3 and syllabus_event/3. Facts are de-duplicated with a set and written in sorted order. rules.pl defines query predicates

(teaches_course, professor_office_hours, office_hours_on_day, course_meeting, course_prerequisite, and others) on top of those facts.

ask_questions.py loads both files into SWI-Prolog via PySwip and matches user questions against regular-expression patterns. User text is escaped into quoted Prolog atoms before being placed in a query. Supported question forms include:

- Who teaches COSC 369?
- What classes does Adedoyin, Anthony teach?
- When are Dr. Appolo Tankeh's office hours?
- Which office hours are on Wednesday?
- Where is Dr. Appolo Tankeh's office?
- What department is <name> in?
- What is the email for Dr. Appolo Tankeh?
- What are the prerequisites for COSC 369?
- What is the attendance policy for COSC 369?
- What is the course description for COSC 369?
- When is the midterm for COSC 369?
- When does COSC 369 meet?

Only syllabus information that was actually provided (COSC 369, Dr. Appolo Tankeh) is added; the system does not invent syllabus data for other courses. Office lookups check syllabus facts first and fall back to the faculty roster.

10. Continuous Integration

.gitlab-ci.yml defines a three-stage pipeline on the python:3.12-bookworm image, which already includes gcc and make:

Stage	What it checks	Artifacts
build	make all with -Werror (any warning fails); lib/libschedule.a and bin/scheduler exist	lib/, bin/
schedule	Installs openpyxl, runs make run, checks all four output files are non-empty, prints the report	output/ (incl. .xlsx)
knowledge	Installs SWI-Prolog and requirements, verifies PySwip, runs make demo, checks facts file exists	prolog/generated_facts.pl

make demo is used rather than make ask because the interactive interface waits for keyboard input, which a CI job cannot provide.

11. Observations and Limitations

Reviewing the code and the current output surfaced the following points worth noting:

- **Possible duplicate instructor.** "Bemley", "Bemley, J" and "Bemley, Jesse" are three separate faculty entries (offices CS204–CS206), each with its own 3-hour requirement. If these are the same person, they are being scheduled 9 hours across three offices instead of one combined schedule, and their Tuesday/Thursday blocks overlap each other.
- **Working-window is not applied between classes.** The 8:00 AM – 5:00 PM window bounds the before-first-class and after-last-class gaps, but a gap between two evening classes is used as-is. For example, Agada, Ruth has blocks at 5:50–7:00 PM and 5:50–6:40 PM, outside the stated window.
- **Short fragments.** A few blocks are much shorter than the 2-hour default (e.g. Wilson, Marcella J. Thursday 9:20–10:00 AM), because the last remaining minutes are placed in the next available gap.
- **Uneven spread across the week.** Because days are scanned Monday first, hours tend to concentrate early in the week; several instructors have nothing Thursday or Friday even when they teach on Thursday.
- **Fallback scope.** Fallback blocks always start at 8:00 AM and are only placed on non-teaching days; if an instructor teaches every preferred day and the gaps are insufficient, the requirement may go unmet (main reports this as a warning).
- **Rooms are not checked.** The scheduler prevents an instructor's own conflicts but does not check classroom or office room conflicts between instructors.
- **Fixed capacities.** Limits of 200 courses, 100 faculty and 20 blocks per instructor are compile-time constants; excess input lines are skipped with a warning.
- **Question parsing.** The question interface uses fixed regular-expression patterns, so questions must closely match the supported phrasings, and names must match the data exactly (e.g. "Adedoyin, Anthony").

12. Recommendations for Future Work

- Normalize instructor names (or add a faculty ID) so the same person cannot appear as multiple entries.
- Clamp all office-hour blocks to the configured working window and make the window and block size configurable from the faculty file.
- Distribute hours across teaching days (e.g. round-robin) rather than front-loading Monday and Tuesday, and enforce a minimum block length.
- Add room-conflict checking for office rooms and classrooms.
- Add unit tests for the C helpers (time parsing, day masks, gap computation) and run them in CI.
- Extend the knowledge base with more syllabi and add fuzzy name matching to the question interface.

13. Conclusion

The project meets its core objectives: a modular C program, built as a static library with a Makefile, reads the class and faculty input files and generates conflict-free office-hour schedules that satisfy every faculty member's weekly requirement (118 of 118 hours scheduled across 27 entries). The Python and Prolog extensions turn that output into a formatted Excel workbook and a queryable knowledge base,

and the GitLab pipeline verifies the full C → Python → Prolog chain on every push. The observations in Section 11 – chiefly the duplicated Bemley entries and the working-window inconsistency – are the most valuable next fixes.